

JAVA OOP PRACTICAL MANUAL

Object-Oriented Programming with Java

Complete Engineering Practical Journal | 10 Practicals

INDEX

- 01. Class and Object Basics
- 02. E-Commerce Order Processing using Constructors and ArrayList
- 03. Method Overloading using Math Class
- 04. Array Operations Program
- 05. Single Inheritance Program
- 06. Interface and Runtime Polymorphism
- 07. Abstract Class Program
- 08. ATM Simulation using Exception Handling
- 09. Employee CRUD using JDBC
- 10. Multithreading Program

PRACTICAL NO. 01

Class and Object Basics

AIM:

To understand and implement the fundamental concepts of classes and objects in Java by defining a class with data members and methods, creating objects, and accessing class members using the dot operator.

OBJECTIVES:

- Understand the concept of a class as a blueprint for objects.
- Declare instance variables and methods inside a class.
- Create objects using the new keyword and understand heap memory allocation.
- Access class members using the dot (.) operator.
- Understand the difference between a class and an object.
- Practice writing a basic Java program using OOP structure.

THEORY:

A Class is a user-defined blueprint that defines fields (data) and methods (behavior). It does not occupy memory by itself. An Object is an instance of a class created at runtime using the new keyword, which allocates memory on the heap. Each object has its own copy of instance variables. Instance variables are declared inside the class but outside any method. Methods define the behavior of the class. The dot (.) operator is used to access fields and call methods on an object. Access modifiers (public, private, protected) control member visibility. The this keyword refers to the current object and is used to resolve naming conflicts between instance variables and parameters.

CONCEPTS USED:

- **Class:** Template defining fields and methods; no memory allocated until object is created.
- **Object:** Instance of a class created using new; stored in heap memory.
- **Instance Variable:** Field declared in a class; each object has its own copy.
- **Method:** Named block of code defining object behavior.
- **new Keyword:** Allocates heap memory and invokes the constructor.
- **Dot Operator (.):** Used to access fields/methods of an object.

SOURCE CODE:

```
class Student {
    String name;
    int rollNo;
    float marks;

    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks: " + marks);
        System.out.println(marks >= 40 ? "Result: PASS" : "Result: FAIL");
    }
}

public class ClassObjectDemo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Rahul Sharma";
        s1.rollNo = 101;
        s1.marks = 78.5f;
        s1.displayInfo();
    }
}
```

OUTPUT:

```
Name: Rahul Sharma
Roll No: 101
Marks: 78.5
Result: PASS
```

RESULT:

The program successfully demonstrated class and object creation in Java. An object of the Student class was instantiated, its fields were assigned values, and the `displayInfo()` method was invoked, producing the correct output.

CONCLUSION:

This practical confirmed that classes and objects are the foundation of Java OOP. A class acts as a blueprint and objects are its real-world instances. Each object maintains its own state and can invoke methods defined in the class.

APPLICATIONS:

- Modeling real-world entities (students, employees, products) in enterprise systems.
- Building GUI applications where each UI component is an object.
- Game development — players, enemies, items as objects.
- Banking systems modeling accounts and customers as objects.
- Web frameworks using objects to represent data models.

PRACTICAL NO. 02

E-Commerce Order Processing using Constructors and ArrayList

AIM:

To simulate a basic e-commerce order processing system using parameterized constructors for object initialization and ArrayList for dynamic storage of multiple order objects.

OBJECTIVES:

- Understand types of constructors (default, parameterized).
- Implement parameterized constructors to initialize object state.
- Use ArrayList from java.util to store objects dynamically.
- Add, iterate, and display objects stored in an ArrayList.
- Apply for-each loop for clean iteration over collections.
- Model a real-world e-commerce scenario using OOP.

THEORY:

A Constructor is a special method with the same name as the class and no return type; it is automatically invoked when an object is created. A Parameterized Constructor accepts arguments to initialize instance variables at creation time, ensuring objects are always in a valid state. The this keyword resolves naming conflicts between parameters and instance variables. ArrayList is a resizable array implementation of the List interface in java.util. Unlike fixed-size arrays, ArrayList grows dynamically. It supports add(), get(), remove(), and size() methods. Generics (e.g., ArrayList<Order>) enforce type safety. The for-each loop iterates over all elements without an explicit index.

CONCEPTS USED:

- **Constructor:** Special method called at object creation to initialize fields.
- **Parameterized Constructor:** Accepts arguments to set field values at creation.
- **ArrayList:** Dynamic, resizable list from java.util for storing objects.
- **Generics:** Type parameter (e.g., <Order>) ensuring type safety in collections.
- **for-each Loop:** Enhanced loop for iterating over collections cleanly.
- **this Keyword:** Refers to current object; resolves parameter-field name conflicts.

SOURCE CODE:

```
import java.util.ArrayList;
class Order {
    int orderId; String customer; String product;
    int qty; double price;
    Order(int id, String c, String p, int q, double pr) {
        this.orderId=id; this.customer=c;
        this.product=p; this.qty=q; this.price=pr;
    }
    void display() {
        System.out.println("Order#"+orderId+" | "+customer+
            " | "+product+" | Qty:"+qty+
            " | Total: Rs."+qty*price));
    }
}
public class ECommerceSystem {
    public static void main(String[] args) {
        ArrayList<Order> orders = new ArrayList<>();
        orders.add(new Order(1001,"Amit","Laptop",1,45000));
        orders.add(new Order(1002,"Sneha","Headphones",2,1500));
        orders.add(new Order(1003,"Ravi","Mouse",3,500));
        System.out.println("=== ORDER LIST ===");
        for (Order o : orders) o.display();
        System.out.println("Total Orders: " + orders.size());
    }
}
```

OUTPUT:

```
=== ORDER LIST ===
Order#1001 | Amit | Laptop | Qty:1 | Total: Rs.45000.0
Order#1002 | Sneha | Headphones | Qty:2 | Total: Rs.3000.0
Order#1003 | Ravi | Mouse | Qty:3 | Total: Rs.1500.0
```

Total Orders: 3

RESULT:

Three Order objects were created using parameterized constructors and stored in an ArrayList. The for-each loop correctly iterated over all orders and displayed their details including computed totals.

CONCLUSION:

Parameterized constructors ensure objects are initialized with valid data immediately upon creation. ArrayList provides flexible, dynamic storage making it ideal for real-world systems where the number of records is variable.

APPLICATIONS:

- Online shopping portals for managing customer orders.
- Inventory management systems to track stock dynamically.
- Hospital systems maintaining patient records.
- Library systems for book borrowing and return tracking.
- Restaurant point-of-sale systems for order management.

PRACTICAL NO. 03

Method Overloading using Math Class

AIM:

To understand and implement method overloading (compile-time polymorphism) in Java, allowing multiple methods with the same name but different parameter lists, and to explore commonly used static methods of the built-in Math class.

OBJECTIVES:

- Understand method overloading and compile-time polymorphism.
- Implement overloaded methods with different parameter counts and types.
- Understand how Java resolves the correct overloaded method at compile time.
- Explore static methods of the java.lang.Math class.
- Differentiate between method overloading and method overriding.
- Apply Math methods: sqrt(), pow(), abs(), max(), min().

THEORY:

Method Overloading allows a class to define multiple methods with the same name but different parameter lists (different number, type, or order of parameters). It is an example of Compile-Time (Static) Polymorphism — the correct method is selected by the compiler based on the method signature. The return type alone cannot distinguish overloaded methods. Java performs automatic type promotion if no exact match is found. The Math class (java.lang, auto-imported) provides static utility methods for mathematical operations. Static methods are called using the class name: Math.sqrt(). Math.PI is a constant equal to 3.14159...

CONCEPTS USED:

- **Method Overloading:** Same method name, different parameter lists in the same class.
- **Compile-Time Polymorphism:** Method resolved at compile time based on argument types.
- **Method Signature:** Method name + parameter list (return type not included).
- **Math Class:** Built-in java.lang class with static mathematical utility methods.
- **Static Method:** Belongs to the class; called via ClassName.method() without an object.
- **Type Promotion:** Java widens argument type if no exact overloaded match is found.

SOURCE CODE:

```
class Calculator {
    int add(int a, int b)          { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
    double add(double a, double b){ return a + b; }
    double area(double r)         { return Math.PI * r * r; }
    double area(double l, double w){ return l * w; }
}

public class OverloadingDemo {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println("add(10,20)      = " + c.add(10, 20));
        System.out.println("add(10,20,30)   = " + c.add(10, 20, 30));
        System.out.println("add(3.5,2.5)    = " + c.add(3.5, 2.5));
        System.out.println("area(circle,r=7)= " + c.area(7));
        System.out.println("area(rect,5x4)  = " + c.area(5, 4));
        System.out.println("Math.sqrt(144)  = " + Math.sqrt(144));
        System.out.println("Math.pow(2,10)   = " + Math.pow(2,10));
        System.out.println("Math.abs(-75)    = " + Math.abs(-75));
        System.out.println("Math.max(45,80)  = " + Math.max(45, 80));
    }
}
```

OUTPUT:

```
add(10,20)      = 30
add(10,20,30)   = 60
add(3.5,2.5)    = 6.0
area(circle,r=7)= 153.94
area(rect,5x4)  = 20.0
Math.sqrt(144)  = 12.0
Math.pow(2,10)  = 1024.0
```

```
Math.abs(-75)    = 75  
Math.max(45,80) = 80
```

RESULT:

All overloaded methods were correctly invoked by the compiler based on argument types and counts. Math class static methods executed accurately without requiring object instantiation.

CONCLUSION:

Method overloading enhances code readability by allowing related operations to share a name. The Java compiler resolves overloaded calls at compile time. The Math class provides a rich set of ready-to-use mathematical functions, reducing the need for custom implementations.

APPLICATIONS:

- Scientific calculators performing operations on multiple data types.
- Graphics libraries with draw() methods for different shapes.
- Logging frameworks accepting String, int, or Exception arguments.
- Java's own String.valueOf() overloaded for all primitive types.
- Financial systems with compute() methods for different interest types.

PRACTICAL NO. 04

Array Operations Program

AIM:

To understand and implement array operations in Java including declaration, initialization, traversal, sorting, searching, and finding maximum, minimum, and sum of elements.

OBJECTIVES:

- Declare and initialize single-dimensional arrays in Java.
- Traverse arrays using for and for-each loops.
- Perform operations: sum, average, maximum, minimum.
- Sort an array using Arrays.sort() and bubble sort.
- Search an element using linear search.
- Understand how arrays are stored in memory.

THEORY:

An Array is a fixed-size, ordered collection of elements of the same data type stored in contiguous memory locations. In Java, arrays are objects allocated on the heap. They are declared as `dataType[] arrayName` and created using `new dataType[size]`. Array indices start at 0. Accessing an index outside the valid range throws `ArrayIndexOutOfBoundsException`. The length property gives the number of elements. Java's Arrays class (`java.util.Arrays`) provides utility methods like `sort()` and `binarySearch()`. Multi-dimensional arrays (2D arrays) are arrays of arrays. Bubble Sort compares adjacent elements and swaps them if they are out of order, repeated for $n-1$ passes.

CONCEPTS USED:

- **Array:** Fixed-size, same-type element collection; zero-indexed; stored on heap.
- **Array Declaration:** `int[] arr = new int[5];` allocates 5 integers.
- **Array Length:** `arr.length` returns the number of elements in the array.
- **Arrays Class:** `java.util.Arrays` provides `sort()`, `binarySearch()`, `fill()` methods.
- **Bubble Sort:** Compares adjacent elements and swaps; $O(n^2)$ time complexity.
- **ArrayIndexOutOfBoundsException:** Runtime error when accessing an invalid index.

SOURCE CODE:

```
import java.util.Arrays;
public class ArrayOperations {
    public static void main(String[] args) {
        int[] arr = {45, 12, 78, 34, 56, 23, 90, 7};
        System.out.println("Original: " + Arrays.toString(arr));
        int sum = 0, max = arr[0], min = arr[0];
        for (int x : arr) {
            sum += x;
            if (x > max) max = x;
            if (x < min) min = x;
        }
        System.out.println("Sum: "+sum+" | Avg: "+(sum/arr.length));
        System.out.println("Max: "+max+" | Min: "+min);
        Arrays.sort(arr);
        System.out.println("Sorted: " + Arrays.toString(arr));
        int key = 56;
        int idx = Arrays.binarySearch(arr, key);
        System.out.println("Search "+key+": index "+idx);
    }
}
```

OUTPUT:

```
Original: [45, 12, 78, 34, 56, 23, 90, 7]
Sum: 345 | Avg: 43
Max: 90 | Min: 7
Sorted: [7, 12, 23, 34, 45, 56, 78, 90]
Search 56: index 5
```

RESULT:

Array operations were successfully performed. Sum, average, maximum, and minimum values were computed. The array was sorted using `Arrays.sort()` and binary search correctly located element 56 at index 5.

CONCLUSION:

Arrays provide efficient random access to elements using indices. Java's `Arrays` utility class simplifies common operations like sorting and searching. Understanding arrays is essential as they form the basis for more advanced data structures.

APPLICATIONS:

- Storing and processing student marks or test scores.
- Image processing where pixels are stored in 2D arrays.
- Matrix operations in scientific computing.
- Implementing stacks and queues using arrays.
- Sorting and searching large datasets in database systems.

PRACTICAL NO. 05

Single Inheritance Program

AIM:

To understand and implement single inheritance in Java where a subclass inherits properties and methods from a single superclass, demonstrating the use of the extends keyword, method inheritance, and method overriding.

OBJECTIVES:

- Understand the concept and benefit of inheritance in OOP.
- Use the extends keyword to create a subclass from a superclass.
- Access inherited fields and methods in the subclass.
- Implement method overriding to provide specialized behavior.
- Use the super keyword to call the superclass constructor and methods.
- Understand IS-A relationship between subclass and superclass.

THEORY:

Inheritance is an OOP mechanism that allows a subclass to acquire the fields and methods of a superclass. In Java, single inheritance is implemented using the extends keyword: `class SubClass extends SuperClass`. The subclass inherits all non-private members of the superclass. The super keyword calls the superclass constructor (`super()`) or method (`super.methodName()`). Method Overriding occurs when a subclass redefines a method with the same name and signature as in the superclass, providing specialized behavior. The `@Override` annotation is used for clarity and compile-time checking. Inheritance promotes code reuse and establishes an IS-A relationship (a Dog IS-A Animal).

CONCEPTS USED:

- **Inheritance:** Mechanism for a subclass to acquire fields/methods of a superclass.
- **extends Keyword:** Used to establish inheritance between two classes.
- **super Keyword:** Calls superclass constructor or method from the subclass.
- **Method Overriding:** Subclass redefines a superclass method with the same signature.
- **@Override Annotation:** Compile-time check that a method is correctly overriding.
- **IS-A Relationship:** Subclass is a type of the superclass (Dog IS-A Animal).

SOURCE CODE:

```
class Animal {
    String name;
    Animal(String name) { this.name = name; }
    void eat() {
        System.out.println(name + " is eating.");
    }
    void sound() {
        System.out.println(name + " makes a sound.");
    }
}

class Dog extends Animal {
    String breed;
    Dog(String name, String breed) {
        super(name);
        this.breed = breed;
    }
    @Override
    void sound() {
        System.out.println(name + " barks! Breed: " + breed);
    }
    void fetch() {
        System.out.println(name + " fetches the ball.");
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog("Bruno", "Labrador");
        d.eat();        // inherited from Animal
        d.sound();       // overridden in Dog
        d.fetch();       // Dog's own method
    }
}
```

OUTPUT:

```
Bruno is eating.  
Bruno barks! Breed: Labrador  
Bruno fetches the ball.
```

RESULT:

The Dog subclass successfully inherited the eat() method from Animal, overrode the sound() method with its own implementation, and added a new fetch() method. The super() constructor correctly initialized the name field.

CONCLUSION:

Single inheritance in Java enables code reuse by allowing subclasses to inherit members from a superclass. Method overriding provides runtime flexibility by allowing subclasses to specialize inherited behavior. The super keyword maintains access to the superclass members.

APPLICATIONS:

- Modeling vehicle hierarchy (Vehicle → Car → ElectricCar).
- GUI frameworks where all UI components inherit from a base Component class.
- Banking systems with Account as superclass and SavingsAccount/LoanAccount as subclasses.
- Employee management with Employee → Manager → Director hierarchy.
- Game development with base Character class and specialized Player/Enemy subclasses.

PRACTICAL NO. 06

Interface and Runtime Polymorphism

AIM:

To understand and implement interfaces in Java as contracts for behavior, and to demonstrate runtime polymorphism (dynamic method dispatch) where the correct overridden method is determined at runtime through superclass/interface references.

OBJECTIVES:

- Understand the purpose and syntax of interfaces in Java.
- Implement an interface in multiple classes.
- Understand runtime (dynamic) polymorphism and dynamic dispatch.
- Use superclass/interface references to hold subclass objects.
- Differentiate between compile-time and runtime polymorphism.
- Understand the role of the implements keyword.

THEORY:

An Interface in Java is a fully abstract contract that defines method signatures without implementation (prior to Java 8). Classes use the implements keyword to promise they will provide implementations for all interface methods. A class can implement multiple interfaces (solving Java's lack of multiple class inheritance). Runtime Polymorphism (Dynamic Method Dispatch) occurs when an overridden method is called through a superclass or interface reference. The JVM determines the correct method to execute at runtime based on the actual object type, not the reference type. This is also called late binding. The @Override annotation is recommended when implementing interface methods.

CONCEPTS USED:

- **Interface:** A contract declaring method signatures; implemented using implements keyword.
- **implements Keyword:** Used by a class to declare it fulfills an interface contract.
- **Runtime Polymorphism:** Method resolution at runtime based on actual object type.
- **Dynamic Dispatch:** JVM selects overridden method based on runtime object type.
- **Late Binding:** Method call resolved at runtime (opposite of compile-time binding).
- **Multiple Interfaces:** A class can implement multiple interfaces simultaneously.

SOURCE CODE:

```
interface Shape {
    double area();
    void display();
}

class Circle implements Shape {
    double radius;
    Circle(double r) { this.radius = r; }
    public double area() { return Math.PI * radius * radius; }
    public void display() {
        System.out.printf("Circle | Radius: %.1f | Area: %.2f\n",
            radius, area());
    }
}

class Rectangle implements Shape {
    double length, width;
    Rectangle(double l, double w) { length=l; width=w; }
    public double area() { return length * width; }
    public void display() {
        System.out.printf("Rect    | %gx%g | Area: %.2f\n",
            length, width, area());
    }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(7), new Rectangle(5, 4) };
        for (Shape s : shapes) s.display(); // runtime dispatch
    }
}
```

OUTPUT:

```
Circle | Radius: 7.0 | Area: 153.94  
Rect   | 5.00x4.00 | Area: 20.00
```

RESULT:

Both Circle and Rectangle implemented the Shape interface. When display() was called through Shape references in the array, the JVM correctly dispatched to each class's own implementation at runtime, demonstrating dynamic method dispatch.

CONCLUSION:

Interfaces enforce a common contract across unrelated classes while allowing each to provide its own implementation. Runtime polymorphism enables flexible, extensible designs where new classes can be added without modifying existing code that uses the interface reference.

APPLICATIONS:

- Payment gateways implementing a common Payment interface (PayPal, Stripe, etc.).
- Sorting algorithms implementing a Comparable or Comparator interface.
- JDBC drivers implementing the Connection interface for different databases.
- Plugin architectures where plugins implement a standard interface.
- UI frameworks with Drawable or Clickable interfaces for components.

PRACTICAL NO. 07

Abstract Class Program

AIM:

To understand and implement abstract classes in Java, which define a partial implementation with one or more abstract methods that must be overridden by concrete subclasses, enforcing a common structure while deferring specific behavior.

OBJECTIVES:

- Understand the purpose of abstract classes and abstract methods.
- Use the abstract keyword for class and method declarations.
- Implement concrete subclasses that override abstract methods.
- Understand that abstract classes cannot be instantiated directly.
- Differentiate between abstract classes and interfaces.
- Understand partial implementation and template method pattern.

THEORY:

An Abstract Class is declared with the abstract keyword and cannot be instantiated directly. It may contain both abstract methods (declared without a body, ending in semicolon) and concrete methods (with full implementation). Abstract methods must be overridden by all non-abstract subclasses. Abstract classes are used when related classes share common code but each requires a specialized implementation of certain methods. Unlike interfaces, abstract classes can have constructors, instance variables, and fully implemented methods. They represent an IS-A relationship. A class that extends an abstract class but does not override all abstract methods must itself be declared abstract.

CONCEPTS USED:

- **Abstract Class:** Class declared with abstract; cannot be instantiated directly.
- **Abstract Method:** Method with no body declared in abstract class; must be overridden.
- **Concrete Subclass:** Non-abstract class that implements all inherited abstract methods.
- **abstract Keyword:** Used to declare both abstract classes and abstract methods.
- **Partial Implementation:** Abstract class provides common code; subclasses provide specifics.
- **Template Pattern:** Abstract class defines structure; subclasses fill in details.

SOURCE CODE:

```
abstract class Vehicle {
    String brand;
    int speed;
    Vehicle(String b, int s) { brand = b; speed = s; }
    abstract void fuelType();    // abstract method
    void showDetails() {        // concrete method
        System.out.println("Brand: "+brand+" | Speed: "+speed+" km/h");
    }
}

class Car extends Vehicle {
    Car(String b, int s) { super(b, s); }
    public void fuelType() {
        System.out.println("Fuel: Petrol/Diesel");
    }
}

class ElectricBike extends Vehicle {
    ElectricBike(String b, int s) { super(b, s); }
    public void fuelType() {
        System.out.println("Fuel: Electric Battery");
    }
}

public class AbstractDemo {
    public static void main(String[] args) {
        Vehicle[] v = { new Car("Toyota",180),
                        new ElectricBike("Ather",90) };
        for (Vehicle x : v) { x.showDetails(); x.fuelType(); System.out.println(); }
    }
}
```

OUTPUT:

```
Brand: Toyota | Speed: 180 km/h  
Fuel: Petrol/Diesel
```

```
Brand: Ather | Speed: 90 km/h  
Fuel: Electric Battery
```

RESULT:

The abstract Vehicle class could not be instantiated. Car and ElectricBike subclasses implemented the abstract fuelType() method. The concrete showDetails() method was shared and reused by both subclasses.

CONCLUSION:

Abstract classes provide a powerful mechanism to enforce a contract while sharing common implementation. They are ideal when related classes need shared code with specialized behavior. The combination of abstract and concrete methods eliminates code duplication.

APPLICATIONS:

- Defining base classes in GUI frameworks (abstract Widget class).
- Template method pattern in game engines (abstract update() in GameObject).
- Database layer with abstract execute() in a base Query class.
- Report generation with abstract generateReport() for PDF, Excel formats.
- Vehicle simulation systems with abstract move() for different transport types.

PRACTICAL NO. 08

ATM Simulation using Exception Handling

AIM:

To simulate an ATM machine in Java demonstrating exception handling using try-catch-finally blocks and custom exception classes to gracefully manage runtime errors like insufficient balance and invalid PIN.

OBJECTIVES:

- Understand the need for exception handling in Java programs.
- Use try-catch-finally blocks to handle runtime exceptions.
- Create and throw custom exception classes.
- Understand checked vs. unchecked exceptions.
- Use the throws keyword in method declarations.
- Demonstrate graceful error handling without program crash.

THEORY:

An Exception is an unexpected event during program execution that disrupts the normal flow. Java's exception handling mechanism uses try-catch-finally blocks. Code that might throw an exception is placed in the try block. The catch block handles specific exception types. The finally block executes always, regardless of whether an exception occurred — typically used for cleanup. Custom Exceptions are created by extending the Exception class (checked) or RuntimeException (unchecked). The throw keyword manually raises an exception; throws declares that a method may throw an exception. The Java exception hierarchy has Throwable at the top, with Error and Exception as subclasses. Common exceptions include NullPointerException, ArrayIndexOutOfBoundsException, and ArithmeticException.

CONCEPTS USED:

- **Exception:** Runtime event disrupting normal program flow; must be handled.
- **try-catch-finally:** Structured blocks for handling and cleaning up after exceptions.
- **Custom Exception:** User-defined exception class extending Exception or RuntimeException.
- **throw:** Keyword to manually raise an exception object.
- **throws:** Declares which exceptions a method may propagate to its caller.
- **Checked/Unchecked:** Checked exceptions must be handled; unchecked (RuntimeException) are optional.

SOURCE CODE:

```
class InsufficientFundsException extends Exception {
    InsufficientFundsException(String msg) { super(msg); }
}
class ATM {
    private double balance;
    private int pin;
    ATM(double bal, int pin) { this.balance=bal; this.pin=pin; }
    void withdraw(int enteredPin, double amount)
        throws InsufficientFundsException {
        try {
            if (enteredPin != pin)
                throw new RuntimeException("Invalid PIN!");
            if (amount > balance)
                throw new InsufficientFundsException(
                    "Insufficient funds! Balance: Rs."+balance);
            balance -= amount;
            System.out.println("Withdrawn: Rs."+amount);
            System.out.println("Remaining: Rs."+balance);
        } catch (InsufficientFundsException e) {
            System.out.println("ERROR: " + e.getMessage());
        } finally {
            System.out.println("Transaction complete.");
        }
    }
}
public class ATMSimulation {
    public static void main(String[] args) {
        ATM atm = new ATM(5000, 1234);
        try { atm.withdraw(1234, 2000); } catch (Exception e){}
        System.out.println();
    }
}
```



```
        try { atm.withdraw(1234, 8000); } catch(Exception e){}
    }
}
```

OUTPUT:

```
Withdrawn: Rs.2000.0
Remaining: Rs.3000.0
Transaction complete.
```

```
ERROR: Insufficient funds! Balance: Rs.3000.0
Transaction complete.
```

RESULT:

The ATM simulation correctly handled a successful withdrawal and an insufficient funds scenario. The custom `InsufficientFundsException` was thrown and caught gracefully. The finally block executed in both cases.

CONCLUSION:

Exception handling in Java prevents program crashes by gracefully managing runtime errors. Custom exceptions make code self-documenting and domain-specific. The finally block ensures critical cleanup code always runs, regardless of the exception outcome.

APPLICATIONS:

- Banking applications handling transaction failures and authentication errors.
- File I/O operations handling `FileNotFoundException` and `IOException`.
- Network applications handling connection timeouts and socket errors.
- Form validation throwing custom exceptions for invalid user input.
- E-commerce systems handling payment gateway failures gracefully.

PRACTICAL NO. 09

Employee CRUD using JDBC

AIM:

To implement basic database operations (Create, Read, Update, Delete) on an Employee table using Java Database Connectivity (JDBC) to demonstrate how Java programs interact with relational databases.

OBJECTIVES:

- Understand the JDBC architecture and its key components.
- Load a JDBC driver and establish a database connection.
- Execute SQL statements using Statement and PreparedStatement.
- Perform INSERT, SELECT, UPDATE, and DELETE operations.
- Process ResultSet to retrieve and display query results.
- Close database resources properly to prevent leaks.

THEORY:

JDBC (Java Database Connectivity) is a Java API that enables Java programs to connect and interact with relational databases. The JDBC architecture consists of: JDBC API (java.sql package), JDBC Driver Manager, and JDBC Drivers (specific to each database). The key steps in JDBC are: Load Driver (Class.forName()), Establish Connection (DriverManager.getConnection()), Create Statement (connection.createStatement() or prepareStatement()), Execute SQL (executeUpdate() for DML, executeQuery() for SELECT), Process ResultSet, and Close Resources. PreparedStatement is preferred over Statement because it prevents SQL injection, supports parameterized queries, and offers better performance through query pre-compilation. ResultSet provides a cursor to iterate through query results row by row.

CONCEPTS USED:

- **JDBC:** Java API (java.sql) for connecting Java programs to relational databases.
- **Connection:** Object representing a live database session; from DriverManager.getConnection().
- **Statement:** Object to execute SQL; PreparedStatement is parameterized and safer.
- **ResultSet:** Cursor-based object to iterate through SELECT query results.
- **PreparedStatement:** Precompiled SQL with placeholders; prevents SQL injection.
- **CRUD:** Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE) operations.

SOURCE CODE:

```
import java.sql.*;

public class EmployeeCRUD {
    static final String URL = "jdbc:mysql://localhost/company";
    static final String USER = "root", PASS = "password";
    public static void main(String[] args) throws Exception {
        Connection con = DriverManager.getConnection(URL,USER,PASS);
        Statement st = con.createStatement();
        // CREATE table
        st.execute("CREATE TABLE IF NOT EXISTS emp"+
            "(id INT PRIMARY KEY, name VARCHAR(50), salary DOUBLE)");
        // INSERT
        st.execute("INSERT INTO emp VALUES(1,'Amit Kumar',55000)");
        st.execute("INSERT INTO emp VALUES(2,'Priya Joshi',62000)");
        System.out.println("Records inserted.");
        // SELECT
        ResultSet rs = st.executeQuery("SELECT * FROM emp");
        System.out.println("ID | Name | Salary");
        while(rs.next())
            System.out.printf("%-4d| %-13s| %.0f%n",
                rs.getInt(1),rs.getString(2),rs.getDouble(3));
        // UPDATE
        st.execute("UPDATE emp SET salary=70000 WHERE id=1");
        System.out.println("Salary updated for id=1.");
        // DELETE
        st.execute("DELETE FROM emp WHERE id=2");
        System.out.println("Record deleted for id=2.");
        con.close();
    }
}
```

OUTPUT:

```
Records inserted.  
ID   | Name       | Salary  
1    | Amit Kumar | 55000  
2    | Priya Joshi | 62000  
Salary updated for id=1.  
Record deleted for id=2.
```

RESULT:

All four CRUD operations were successfully executed. The Employee table was created, two records were inserted and displayed via SELECT, one record was updated, and one was deleted using JDBC with a MySQL database.

CONCLUSION:

JDBC provides a standardized way for Java programs to communicate with any JDBC-compliant database. Understanding CRUD operations through JDBC is essential for building data-driven enterprise applications. PreparedStatement should always be preferred for security and performance.

APPLICATIONS:

- Employee management systems with database-backed CRUD operations.
- Online examination systems storing questions and student responses.
- Library management for tracking books and members.
- E-commerce backends managing products, orders, and customers.
- Hospital management systems maintaining patient and appointment records.

PRACTICAL NO. 10

Multithreading Program

AIM:

To understand and implement multithreading in Java using the Thread class and Runnable interface, demonstrating concurrent execution of multiple threads, thread lifecycle, and synchronization.

OBJECTIVES:

- Understand the concept of multithreading and its advantages.
- Create threads by extending the Thread class.
- Create threads by implementing the Runnable interface.
- Understand the thread lifecycle (New, Runnable, Running, Blocked, Dead).
- Use sleep() to simulate concurrent task execution.
- Understand basic thread synchronization concepts.

THEORY:

Multithreading is the ability of a CPU to execute multiple threads concurrently, allowing a program to perform multiple tasks simultaneously. A Thread is the smallest unit of execution within a process. Java supports multithreading natively through the java.lang.Thread class and the Runnable interface. To create a thread via Thread class, extend Thread and override run(). To use Runnable, implement the run() method and pass the Runnable to a Thread constructor. Thread lifecycle states: New (created, not started), Runnable (ready to run), Running (executing), Blocked/Waiting (waiting for a resource or another thread), and Terminated (finished). Thread.sleep(ms) pauses a thread for specified milliseconds. Synchronization uses the synchronized keyword to prevent race conditions when multiple threads share resources.

CONCEPTS USED:

- **Thread:** Lightweight unit of execution sharing process memory; runs concurrently.
- **Runnable Interface:** Functional interface with run() method; preferred for thread creation.
- **Thread Lifecycle:** New → Runnable → Running → Blocked/Waiting → Terminated.
- **Thread.sleep():** Pauses current thread for given milliseconds; throws InterruptedException.
- **Synchronization:** synchronized keyword prevents concurrent access to shared resources.
- **Race Condition:** Bug caused when multiple threads modify shared data without synchronization.

SOURCE CODE:

```
class PrintNumbers extends Thread {
    String threadName;
    PrintNumbers(String name) { this.threadName = name; }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(threadName + " -> " + i);
            try { Thread.sleep(300); }
            catch (InterruptedException e) { e.printStackTrace(); }
        }
    }
}

class PrintLetters implements Runnable {
    public void run() {
        for (char c = 'A'; c <= 'E'; c++) {
            System.out.println("Letters -> " + c);
            try { Thread.sleep(400); }
            catch (InterruptedException e) { e.printStackTrace(); }
        }
    }
}

public class MultithreadingDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new PrintNumbers("Thread-1");
        Thread t2 = new Thread(new PrintLetters());
        t1.start();
        t2.start();
        t1.join(); t2.join();
        System.out.println("All threads completed.");
    }
}
```

OUTPUT:

```
Thread-1 -> 1
Letters -> A
Thread-1 -> 2
Letters -> B
Thread-1 -> 3
Letters -> C
Thread-1 -> 4
Letters -> D
Thread-1 -> 5
Letters -> E
All threads completed.
(Note: Output order may vary due to thread scheduling)
```

RESULT:

Both threads ran concurrently. Thread-1 (extending Thread) printed numbers 1-5, while the Runnable-based thread printed letters A-E simultaneously. The join() calls ensured main() waited for both threads to finish.

CONCLUSION:

Multithreading improves application performance by executing multiple tasks in parallel. Java supports thread creation via Thread class extension and Runnable implementation. Understanding the thread lifecycle and synchronization is critical for writing safe, efficient concurrent programs.

APPLICATIONS:

- Web servers handling multiple client requests simultaneously.
- Media players running audio, video, and UI threads concurrently.
- Download managers downloading multiple files in parallel.
- Real-time data processing systems with producer-consumer patterns.
- Video games running physics, rendering, and input handling in separate threads.